

DKNexus: Diseño e Implementación de un Lenguaje de Dominio Específico para Machine Learning y Deep Learning

Darien Esteban Parra Guzmán

Resumen

Este trabajo presenta **DKNexus**, un Lenguaje de Dominio Específico (DSL) diseñado e implementado desde cero para expresar y entrenar modelos de Machine Learning (ML) y Deep Learning (DL). El frontend se construye con ANTLR 4 (análisis léxico y sintáctico) y el backend es un intérprete en Python basado en el patrón Visitor. La premisa central del proyecto es la *restricción estricta de cero librerías externas de cálculo*: la trigonometría se obtiene por series de Taylor, el álgebra lineal se implementa sobre un núcleo numérico propio en C (“DKNumpy”) con memoria contigua y *binding* mediante `ctypes`, y los algoritmos de aprendizaje (regresión lineal, regresión logística, perceptrón, perceptrón multicapa, k-medias y k-vecinos) se programan manualmente. El runtime incorpora un heap simulado con punteros y ámbitos, un guardián de ejecución contra bucles infinitos, persistencia clave-valor mediante Redis con *fallback* a disco, y carga de archivos CSV reales. Los experimentos validan la corrección de los modelos: la red multicapa resuelve el problema no lineal XOR con exactitud 1,0, y el sistema procesa un conjunto de datos real de 10 280 registros calculando estadísticas, regresión y agrupamiento.

Index Terms

Lenguaje de dominio específico, ANTLR, intérprete, Visitor, Machine Learning, Deep Learning, perceptrón multicapa, descenso de gradiente, `ctypes`, memoria contigua.

I. INTRODUCCIÓN

LOS marcos de trabajo modernos de aprendizaje automático, como NumPy y scikit-learn, ocultan al programador la mecánica interna del cálculo numérico y de los algoritmos de optimización. Si bien esto es deseable en producción, limita la comprensión profunda de *cómo* funcionan dichos sistemas. El presente proyecto adopta el enfoque contrario: construir un lenguaje de programación completo, denominado **DKNexus**, en el que cada componente —desde la función seno hasta la retropropagación de una red neuronal— se implementa de forma explícita.

El objetivo académico, enmarcado en la asignatura de Lenguajes de Programación y Transducción, abarca tres cortes: (i) el diseño del lenguaje (gramática, léxico, sintaxis y semántica); (ii) un lenguaje de propósito general (ciclos, funciones, recursividad, archivos); y (iii) la parte fuerte de aprendizaje profundo (regresión, clasificación, agrupamiento, métricas y visualización). Este documento describe la solución integrada y presenta los resultados experimentales que validan su correcto funcionamiento.

II. TRABAJO DE REFERENCIA

El diseño de DKNexus se apoya en cuatro referencias principales. De **NumPy** se adopta el modelo de *memoria contigua*: las matrices se aplanan en un bloque lineal y se opera con aritmética de punteros. De **scikit-learn** se imita la interfaz de alto nivel `fit/predict` y la familia de modelos (SGDRegressor, Perceptron, LogisticRegression, MLPClassifier, KMeans, KNeighborsClassifier). El libro *Grokking* inspira la formulación iterativa del descenso de gradiente. Finalmente, **ANTLR 4** provee la infraestructura de generación de analizadores y el patrón Visitor.

III. ARQUITECTURA DEL SISTEMA

La ejecución de un programa `.dkn` sigue un *pipeline* de tres etapas, ilustrado en la Figura 1: el *lexer* convierte el texto en *tokens*, el *parser* construye el árbol sintáctico (AST), y el `EvalVisitor` recorre dicho árbol ejecutando cada nodo. El intérprete se apoya en módulos especializados, resumidos en la Tabla I.

D. E. Parra Guzmán cursa Ciencias de la Computación e Inteligencia Artificial en la Universidad Sergio Arboleda, Bogotá, Colombia (e-mail: darien.parra01@usa.edu.co).

Asignatura: PCIA5007-G02:CCV — Lenguajes de Programación y Transducción. Profesor: Joaquín Fernando Sánchez Cifuentes.

Documento entregado el 3 de junio de 2026.

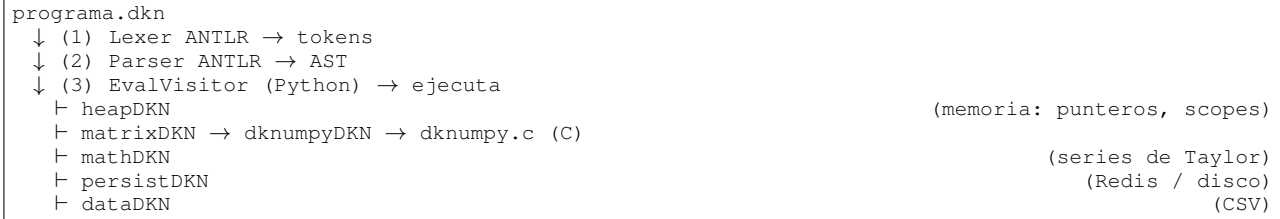


Figura 1. Pipeline de ejecución y módulos del intérprete DKNexus.

Cuadro I
MÓDULOS PRINCIPALES DEL PROYECTO

Archivo	Responsabilidad
grammarDKN.g4	Gramática: tokens y reglas sintácticas.
interpreterDKN.py	Intérprete (EvalVisitor) y REPL.
heapDKN.py	Heap dinámico, punteros y ámbitos.
mathDKN.py	Matemática propia sin import math.
matrixDKN.py	Álgebra de matrices (delega a C).
dknumpy.c	Núcleo numérico nativo en C.
dknumpyDKN.py	Binding ctypes y fallback.
persistDKN.py	Persistencia Redis / disco.
dataDKN.py	Lectura de CSV (numérico y mixto).

IV. DISEÑO DEL LENGUAJE

DKNexus emplea una sintaxis de estilo C, con bloques delimitados por llaves y sentencias terminadas en punto y coma. El listado siguiente muestra las construcciones fundamentales: asignación con inferencia de tipos, control de flujo y funciones recursivas.

```

x = 10;           // entero
v = [1, 2, 3];    // vector
m = [[1, 2], [3, 4]]; // matriz
d = {"lr": 0.01}; // diccionario

if (x > 5) { print("grande"); }
for (i = 0; i < 3; i = i + 1) { print(i); }

function factorial(n) {
  if (n <= 1) { retornar 1; }
  retornar n * factorial(n - 1);
}

```

El analizador reconoce tokens (NUMBER, STRING, VARIABLE), palabras reservadas y operadores con precedencia (^, *, /, %, +, -, comparación, lógicos). La capa semántica valida tipos (p. ej., prohíbe sumar número con cadena), rangos de índices y dominios matemáticos, produciendo mensajes de error legibles.

V. GESTIÓN DE MEMORIA

A diferencia de delegar por completo la memoria al recolector de Python, DKNexus simula un *heap* explícito. Cada variable almacena un **puntero** (dirección hexadecimal) hacia el HeapManager, y una pila de ámbitos gestiona la visibilidad: al invocar una función se apila un ámbito y al retornar se libera, habilitando recursividad con aislamiento.

Para soportar cargas de ML, el heap es **dinámico** (sin tope rígido) y las matrices no se guardan como listas de listas, sino como **bloques contiguos** reservados con malloc en C; el heap solo registra el puntero del bloque. La instrucción `dump_memory()` expone este estado, incluyendo la dirección del bloque nativo y el *backend* activo (C-native o python-fallback).

VI. NÚCLEO NUMÉRICO DKNUMPY

El elemento (i, j) de una matriz de c columnas se localiza mediante aritmética de punteros:

$$\text{índice} = i \cdot c + j. \quad (1)$$

El cómputo pesado se ejecuta en C y se invoca por ctypes: multiplicación (`dknp_matmul`), suma/resta, producto escalar, transpuesta, producto punto y, de forma central para el aprendizaje, la actualización de pesos por descenso de gradiente:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla, \quad (2)$$

donde η es la tasa de aprendizaje. La validación de dimensiones (por ejemplo, que en $A_{2 \times 3} \cdot B_{3 \times 4}$ coincidan las dimensiones internas) se realiza en Python antes de delegar a C. Si la librería nativa no está compilada, un *fallback* en Python puro preserva la funcionalidad.

VII. ALGORITMOS DE MACHINE LEARNING

VII-A. Regresión Lineal

Ajusta $y = w_0 + w_1x$ minimizando el error cuadrático mediante la ecuación (2). El gradiente del sesgo es el error $e = \hat{y} - y$ y el de la pendiente es $e \cdot x$.

VII-B. Clasificación: Perceptrón y Regresión Logística

El perceptrón usa la función escalón de Heaviside; la regresión logística usa la sigmoide

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad (3)$$

con saturación numérica para evitar desbordamiento. Ambos modelos aprenden la compuerta lógica OR con clasificación perfecta.

VII-C. Agrupamiento y Vecindad

`kmeans(X, k, iters)` agrupa de forma no supervisada: asigna cada punto al centroide más cercano (distancia euclídea) y recalcula los centroides como el promedio de sus miembros. La inicialización es determinista (primeros k puntos) para garantizar reproducibilidad. `knn(...)` clasifica por voto mayoritario de los k vecinos más cercanos.

VIII. DEEP LEARNING: PERCEPTRÓN MULTICAPA

El problema XOR no es linealmente separable, por lo que requiere al menos una capa oculta. La red, creada con `mlp_init([2, 4, 1])`, almacena W_1, b_1, W_2, b_2 . El entrenamiento (`mlp_fit`) aplica, por época, un *forward pass*

$$A_1 = \sigma(XW_1 + b_1), \quad A_2 = \sigma(A_1W_2 + b_2), \quad (4)$$

y una retropropagación basada en la regla de la cadena, empleando la derivada de la sigmoide $\sigma'(a) = a(1 - a)$:

$$dZ_2 = A_2 - Y, \quad dW_2 = A_1^\top dZ_2, \quad (5)$$

$$dZ_1 = (dZ_2W_2^\top) \odot A_1 \odot (1 - A_1), \quad dW_1 = X^\top dZ_1. \quad (6)$$

Todo el álgebra se implementa en Python puro y se protege con el guardián de ejecución. El listado siguiente resume el uso del modelo.

```
X = [[0.0, 0.0], [0.0, 1.0], [1.0, 0.0], [1.0, 1.0]];
Y = [[0.0], [1.0], [1.0], [0.0]];
red = mlp_init([2, 4, 1]);
red = mlp_fit(red, X, Y, 0.5, 5000);
print(mlp_predict(red, X)); // [0.0, 1.0, 1.0, 0.0]
```

IX. SERVICIOS DEL RUNTIME

IX-A. Persistencia

`persistDKN` guarda estado clave-valor en Redis (RAM); si no hay servidor disponible, recurre automáticamente a un archivo JSON en disco. Los valores se serializan en JSON, conservando su tipo. Esto permite, por ejemplo, almacenar los pesos de un modelo entrenado.

IX-B. Manejo de Datos

`read_csv` interpreta archivos CSV reales con encabezado, comillas y columnas mixtas (texto y números), devolviendo una tabla con cabecera y filas. Las funciones `csv_col` y `csv_col_num` extraen columnas para análisis estadístico o entrenamiento.

IX-C. Seguridad de Ejecución

Un contador global de instrucciones limita la ejecución; al superar el umbral se lanza un error controlado de *timeout*, evitando que bucles no acotados bloqueen la máquina anfitriona. La jerarquía de errores distingue `DKNParseError`, `DKNRuntimeError` y `DKNMemoryError`.

Cuadro II
RESULTADOS DE LAS PRUEBAS DE LOS MODELOS

Prueba	Métrica	Resultado
Regresión lineal ($y = 2x + 1$)	MSE	$\approx 7 \times 10^{-6}$
Regresión logística (OR)	Exactitud	1,0
Perceptrón (AND)	Exactitud	1,0
MLP (XOR)	Exactitud	1,0
KNN ($k = 3$)	Clasificación	correcta
K-Means ($k = 2$)	Etiquetas	[0, 0, 1, 1]
Precisión	$TP/(TP+FP)$	0,667

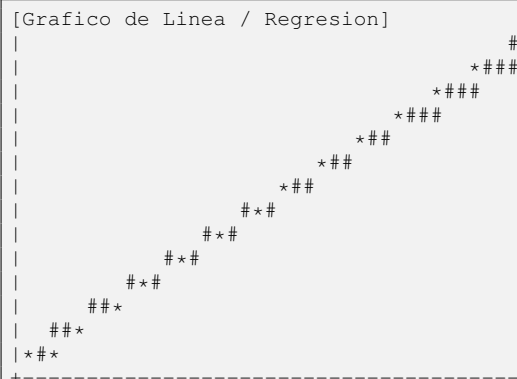
X. RESULTADOS EXPERIMENTALES

La Tabla II resume los resultados obtenidos al ejecutar la batería de pruebas. Todos los modelos alcanzan el comportamiento esperado.

La salida del perceptrón multicapa sobre el XOR, junto con su matriz de confusión perfecta, se reproduce a continuación:

```
Predicciones XOR (esperado 0,1,1,0):
[0.0, 1.0, 1.0, 0.0]
[Matriz de Confusion]
      Pred: 0  Pred: 1
Real 0:    [2]      [0]
Real 1:    [0]      [2]
```

El sistema de visualización genera gráficos en un lienzo ASCII. El siguiente ejemplo muestra los puntos de datos (*) y la recta de regresión aprendida (#):



X-A. *Procesamiento de Datos Reales*

Sobre un archivo `data.csv` de 10 280 registros con seis columnas (*id*, *edad*, *nivel_edu*, *ingreso_mensual*, *sex*, *anios_esc*), el sistema calcula correctamente un ingreso mensual promedio de ≈ 7362 y una edad promedio de $\approx 39,7$ años. Tras normalizar las variables, la regresión lineal converge con un $MSE \approx 0,03$ y el agrupamiento k-medias ($k = 3$) segmenta la muestra sin errores de ejecución.

X-B. Robustez ante Errores

Se verificó que el intérprete reacciona de forma controlada ante entradas inválidas: dimensiones incompatibles, tipos erróneos, parámetros fuera de rango y número incorrecto de argumentos, además de la detección de bucles infinitos por el guardián de ejecución. En todos los casos se emite un mensaje de error descriptivo sin caída del sistema.

XI. CONCLUSIONES

DKNexus demuestra que es viable construir un lenguaje de dominio específico funcional para aprendizaje automático sin recurrir a librerías externas de cálculo, combinando claridad pedagógica con eficiencia mediante un núcleo nativo en C. El proyecto integra los tres cortes de la asignatura: un diseño de lenguaje con gramática formal, un runtime de propósito general con memoria gestionada y recursividad, y un conjunto completo de algoritmos de ML/DL validados experimentalmente.

incluyendo la resolución del problema no lineal XOR mediante una red multicapa con retropropagación. Como trabajo futuro se contemplan funciones de activación adicionales (ReLU, softmax), arquitecturas de red con número variable de capas y métricas complementarias (recall, F1).

REFERENCIAS

- [1] C. R. Harris *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.
- [2] F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [3] T. Parr, *The Definitive ANTLR 4 Reference*, 2nd ed. Pragmatic Bookshelf, 2013.
- [4] A. W. Trask, *Grokking Deep Learning*. Manning Publications, 2019.
- [5] J. L. Carlson, *Redis in Action*. Manning Publications, 2013.

Darien Esteban Parra Guzmán Estudiante de Ciencias de la Computación e Inteligencia Artificial en la Universidad Sergio Arboleda.